

CODING CHALLENGE

Vanilla Policy Gradient & REINFORCE

Objective

Implement a Vanilla Policy Gradient agent using the REINFORCE algorithm and train it on the CartPole-v1 environment. The agent learns a stochastic policy directly from complete episodes and improves its actions using the observed rewards.

Problem Statement

Build a policy-gradient reinforcement learning solution without using a pre-built REINFORCE implementation. The neural network should receive the current CartPole state and output action probabilities. After each episode, calculate discounted returns and update the policy using the REINFORCE objective.

Learning Outcomes

- Understand policy-based reinforcement learning.
- Implement a stochastic policy network with PyTorch.
- Calculate discounted rewards (returns).
- Apply the REINFORCE / Vanilla Policy Gradient update rule.
- Evaluate learning using episode rewards.

Environment

CartPole-v1: the agent moves a cart left or right to keep a pole balanced. The episode ends when the pole falls, the cart moves outside the allowed range, or the time limit is reached.

Expected Result

The average reward should increase during training. A successful agent can approach the CartPole-v1 target of about 500 reward.

1. Theory: Vanilla Policy Gradient

Policy-gradient methods learn a parameterized policy $\pi_{\theta}(a|s)$, where θ represents the parameters of a neural network. Instead of learning a value for every action, the network directly estimates the probability of selecting each action.

REINFORCE Objective

The objective is to maximize expected return:

$$J(\theta) = E[G_t]$$

The gradient estimator used by REINFORCE is:

$$\nabla_{\theta} J(\theta) \approx \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t$$

where G_t is the discounted return from time t :

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

Policy Network

For CartPole, the observation has four values: cart position, cart velocity, pole angle, and pole angular velocity. A small multilayer perceptron maps these four inputs to two action probabilities: move left or move right.

Training Procedure

- Reset the environment and start an episode.
- Observe the state and sample an action from the policy distribution.
- Store the log-probability of the selected action and its reward.
- Continue until the episode terminates.
- Calculate discounted returns for all collected rewards.
- Normalize returns to reduce training instability.
- Compute the policy loss using $-\log \pi(a|s) \times \text{return}$.
- Backpropagate the loss and update the network.

Why REINFORCE?

REINFORCE is one of the simplest policy-gradient algorithms. It is easy to understand and provides a direct connection between the probability of an action and the reward obtained after taking that action. Its main limitation is high variance, so reward normalization and a suitable learning rate can improve stability.

2. Colab Implementation

The following code is designed to run in Google Colab. It installs the required Gymnasium and PyTorch packages, creates the policy network, implements REINFORCE manually, trains the agent, and plots the learning curve.

```
# Vanilla Policy Gradient / REINFORCE - Google Colab
!pip -q install "gymnasium[classic-control]" torch matplotlib

import gymnasium as gym
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.distributions import Categorical
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

class PolicyNetwork(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, action_dim),
            nn.Softmax(dim=-1)
        )

    def forward(self, state):
        return self.net(state)

def discounted_returns(rewards, gamma=0.99):
    returns = []
    G = 0.0
    for reward in reversed(rewards):
        G = reward + gamma * G
        returns.insert(0, G)
    returns = torch.tensor(returns, dtype=torch.float32, device=device)
    if len(returns) > 1:
        returns = (returns - returns.mean()) / (returns.std() + 1e-8)
    return returns

env = gym.make("CartPole-v1")
state_dim = env.observation_space.shape[0]
action_dim = env.action_space.n

policy = PolicyNetwork(state_dim, action_dim).to(device)
optimizer = optim.Adam(policy.parameters(), lr=1e-3)

episodes = 1000
gamma = 0.99
reward_history = []

for episode in range(episodes):
    state, _ = env.reset()
    log_probs, rewards = [], []
    done = False

    while not done:
        state_tensor = torch.tensor(
            state, dtype=torch.float32, device=device
        ).unsqueeze(0)

        probs = policy(state_tensor)
        dist = Categorical(probs)
        action = dist.sample()

        next_state, reward, terminated, truncated, _ = env.step(
            action.item()
        )
```

```

done = terminated or truncated

log_probs.append(dist.log_prob(action))
rewards.append(reward)
state = next_state

returns = discounted_returns(rewards, gamma)

loss = 0
for log_prob, G in zip(log_probs, returns):
    loss += -log_prob * G

optimizer.zero_grad()
loss.backward()
optimizer.step()

total_reward = sum(rewards)
reward_history.append(total_reward)

if (episode + 1) % 50 == 0:
    avg = np.mean(reward_history[-50:])
    print(f"Episode {episode+1:4d} | "
          f"Average Reward: {avg:7.2f}")

env.close()

plt.figure(figsize=(10, 5))
plt.plot(reward_history, alpha=0.35, label="Episode Reward")

window = 50
if len(reward_history) >= window:
    moving_avg = np.convolve(
        reward_history, np.ones(window) / window, mode="valid"
    )
    plt.plot(
        range(window - 1, len(reward_history)),
        moving_avg, linewidth=2, label="50-Episode Average"
    )

plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("REINFORCE Learning Curve - CartPole-v1")
plt.legend()
plt.grid(True)
plt.show()

print("Best episode reward:", max(reward_history))
print("Final 50-episode average:",
      np.mean(reward_history[-50:]))

```

3. Explanation of the Implementation

Policy Network

PolicyNetwork is a feed-forward neural network with two hidden layers. The final Softmax layer converts network outputs into probabilities for the two CartPole actions. Categorical distribution is then used to sample an action according to these probabilities.

Discounted Return

The `discounted_returns` function processes rewards from the end of the episode toward the beginning. Future rewards receive progressively smaller weights according to $\gamma = 0.99$. The resulting returns are normalized before calculating the loss.

REINFORCE Loss

For each selected action, the code stores $\log \pi_{\theta}(a|s)$. The loss is $-\log \pi_{\theta}(a|s)G_t$. Actions followed by high returns receive stronger reinforcement, while actions associated with poor returns are discouraged.

Optimizer

Adam is used to update the policy parameters. The learning rate is 0.001. The optimizer performs gradient descent on the policy loss after each complete episode.

Important Parameters

- Environment: CartPole-v1
- Episodes: 1000
- Discount factor γ : 0.99
- Learning rate: 0.001
- Optimizer: Adam
- Hidden layers: 128 $\rightarrow$ 128
- Action space: 2 actions

Evaluation

The training curve contains the reward obtained in every episode and a 50-episode moving average. Because reinforcement learning is stochastic, the exact reward values can differ between Colab runs. The key indicator is an upward trend in the moving average.

Troubleshooting in Colab

- Run the installation cell first.
- If the notebook was restarted, run all cells from the beginning.
- Do not use the old 'gym' package; this solution uses 'gymnasium'. Different random seeds can produce different learning curves.